LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 06 – Inheritance



Enggar Tri Rahmadhani

103132400007

PS1DS – 05 - REG – 01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

1. Inherintance (pewarisan)

   Inheritance merupakan suatu cara untuk menurunkan suatu kelas yang lebih umum menjadi kelas yang lebih spesifik. Dalam konsep ini dikenal 2 istilah yaitu superclass (kelas dasar/base class) yaitu kelas yang menurunkan, dan subclass (kelas turunan/derived class) yaitu kelas yang diturunkan. Karakteristik (atribut dan method) pada superclass akan dimiliki juga oleh subclass-nya, sehingga inheritance berfungsi menghindari duplikasi objek berupa field, variabel, maupun method. Dalam Java, inheritance diimplementasikan menggunakan keyword extends, dan merepresentasikan hubungan "is a relationship" (adalah bagian dari).

2. Hubungan antar kelas

   Keyword super digunakan untuk mereferensikan superclass dari suatu kelas, yaitu untuk mengakses member (atribut maupun method) milik superclass, serta untuk memanggil konstruktor superclass dari dalam konstruktor subclass.

3. Overloading method

   Overloading method adalah kemampuan untuk membuat dua atau lebih method dengan nama yang sama dalam satu kelas, tetapi jumlah dan tipe data argumennya harus berbeda. Overloading memungkinkan satu nama method melayani berbagai variasi parameter.

4. Overriding method

   Overriding method terjadi apabila dalam suatu subclass didefinisikan sebuah method yang sama (nama, parameter, dan tipe return) dengan method yang dimiliki oleh superclass. Akibatnya, ketika method tersebut dipanggil dari instance subclass, maka method milik subclass-lah yang akan dieksekusi, bukan method milik superclass. Method superclass masih dapat dipanggil dari dalam method overriding dengan menggunakan keyword super.

5. Polymorphism

   Polymorphism merupakan kemampuan sebuah objek untuk membolehkan mengambil beberapa bentuk yang berbeda agar tidak terjadi duplikasi objek. Polymorphism memungkinkan digunakannya suatu interface yang sama untuk memerintah objek agar melakukan aksi yang secara prinsip sama namun secara proses berbeda. Polymorphism erat kaitannya dengan inheritance dan overriding, karena method yang sama dapat berperilaku berbeda tergantung objek yang memanggilnya.

B. Guided

1. gd1

   file: A.java

```java
package com.example.enggar.guided.gd1;

public class A {
    int x;
    int y;

    void tampilXY(){
        System.out.println("Nilai x = "+x+", y = "+y);
    }
}
```

   file: B.java

```java
package com.example.enggar.guided.gd1;

public class B extends A {
    int z;
    int hasil;

    void jumlahXYZ(){
        hasil = (x+y+z);
        System.out.println("Jumlah (xyz) = " + hasil);
    }
}
```

file: Demo.java

```java
package com.example.enggar.guided.gd1;

public class Demo {
    public static void main(String[] args) {
        A superclass = new A();
        B subclass = new B ();

        superclass.x = 3;
        superclass.y = 4;
        System.out.println("Superclass: ");
        superclass.tampilXY();

        subclass.x = 1;
        subclass.y = 2;
        subclass.z = 5;
        System.out.println("subclass: ");
        subclass.jumlahXYZ();
    }
}
```

Output :

```
ar.guided.gd1.Demo'
Superclass:
Nilai x = 3, y = 4
subclass:
Jumlah (xyz) = 8
PS C:\Users\ADVAN WORKPLU
```

2. gd2

file: Employed.java

```java
package com.example.enggar.guided.gd2;

public class Employed {
    String departeman;
    private String nama;

    public Employed(String nama){
        this.nama = nama;
```

```
    }
    public void tampilnama(){
        System.out.println("nama : "+ nama);
    }

}
```

file: Manager.java

```
package com.example.enggar.guided.gd2;

public class Manager extends Employed {
    private String alamat;

    public Manager(String nama, String alamat){
        super(nama);
        this.alamat = alamat;
    }
    public void tampilinformasi(){
        super.tampilnama();
        System.out.println("alamat: " + alamat);
        System.out.println("departemen : " + super.departeman);
    }
}
```

file: Demo.java

```
package com.example.enggar.guided.gd2;

public class Demo {
    public static void main(String[] args) {
        Manager adi = new Manager("adi", "sukabirus");
        adi.departeman = "SDM";
        adi.tampilinformasi();

    }
}
```

Output :

```
in32-x86_64\bin\java.exe' '-XX:+S
ts\coolyeah\sems 5\pbo\modul06\ta
nama : adi
alamat: sukabirus
departemen : SDM
PS C:\Users\ADVAN WORKPLUS\Docume
```

3. gd3

file: Lagu.java

```java
package com.example.enggar.guided.gd3;

public class Lagu {
    String pencipta;
    String judul;

    void isiParam(String param1){
        judul = param1;
        pencipta = "tidak dikenal";

    }

    void isiParam(String param1, String param2){
        judul = param1;
        pencipta = param2;
    }

    void tampilkan(){
        System.out.println("judul : " + judul + "pencipta : " +
pencipta);
    }
}
```

file: Main.java

```java
package com.example.enggar.guided.gd3;

public class Main {
    public static void main(String[] args) {
        Lagu d,a;
        d = new Lagu();
        a = new Lagu();
```

```
        d.isiParam("Lagu 1, ");
        a.isiParam("Lagu 2, ", "aku");

        d.tampilkan();
        a.tampilkan();
    }
}
```

Output :

```
es' '-cp' 'C:\Users\ADVAN WORKPLUS\Documents\cool
ar.guided.gd3.Main'
judul : Lagu 1, pencipta : tidak dikenal
judul : Lagu 2, pencipta : aku
PS C:\Users\ADVAN WORKPLUS\Documents\coolyeah\ser
```

4. gd4

file: A.java

```
package com.example.enggar.guided.gd4;

public class A {
    public void tampilkan() {
        System.out.println("method milik class A dipanggil..");
    }
}
```

file: B.java

```
package com.example.enggar.guided.gd4;

public class B extends A {
    public void tampilkan(){
        super.tampilkan();
        System.out.println("method milik class b dipanggil..");
    }
}
```

file: Main.java

```java
package com.example.enggar.guided.gd4;

public class Main {
    public static void main(String[] args) {
        B overidB = new B();
        overidB.tampilkan();
    }
}
```

Output :

```
in32-x86_64\bin\java.exe' '-XX:+ShowCodeD
ts\coolyeah\sems 5\pbo\modul06\target\cla
method milik class A dipanggil..
method milik class b dipanggil..
PS C:\Users\ADVAN WORKPLUS\Documents\cool
```

C. Unguided

1. Soal Unguided – Modul 6

   Buatlah sebuah program Java untuk menghitung estimasi biaya komputasi dari tiga jenis model machine learning, yaitu model regresi, model klasifikasi, dan model klasifikasi berbasis deep learning. Program ini terdiri dari satu kelas induk dan tiga kelas turunan, di mana seluruh atribut wajib bersifat private, sedangkan penamaan kelas, atribut, dan method dibebaskan.

   a. Kelas induk berfungsi sebagai model umum yang menyimpan nama model dan jumlah data latih. Kelas ini memiliki constructor, method untuk menampilkan informasi, serta dua method hitung biaya dengan nama yang sama (overloading): tanpa parameter yang menghitung data latih dikali 2, dan dengan satu parameter epoch yang menghitung data latih dikali epoch.

   b. Kelas turunan pertama adalah model regresi yang mewarisi kelas induk dan menambahkan atribut jumlah fitur. Kelas ini meng-override method tampilkan informasi dan hitung biaya tanpa parameter dengan rumus biaya induk ditambah jumlah fitur dikali 10. Selain itu, terdapat satu method baru untuk hitung biaya

dengan dua parameter (epoch dan batch) yang menghitung data latih dikali epoch lalu dibagi batch.

c. Kelas turunan kedua adalah model klasifikasi yang mewarisi kelas induk dan menambahkan atribut jumlah kelas. Kelas ini meng-override method tampilkan informasi dan hitung biaya tanpa parameter dengan rumus biaya induk dikali jumlah kelas, namun tidak meng-override method hitung biaya yang memiliki parameter epoch.

d. Kelas turunan ketiga adalah model klasifikasi deep learning yang mewarisi kelas turunan kedua (bukan langsung kelas induk) dan menambahkan atribut jumlah layer. Kelas ini meng-override method tampilkan informasi dan hitung biaya tanpa parameter dengan rumus biaya dari kelas di atasnya ditambah jumlah layer dikali 200.

e. Terakhir, pada class Main, buatlah tiga objek yaitu Regresi Linier (500 data latih, 4 fitur), Decision Tree (800 data latih, 3 kelas), dan Neural Network (1000 data latih, 5 kelas, 3 layer). Simpan ketiga objek tersebut ke dalam sebuah array bertipe kelas induk, lalu lakukan perulangan untuk menampilkan informasi masing-masing model, menghitung biaya komputasinya, dan menjumlahkan seluruh biaya. Setelah perulangan selesai, tampilkan total biaya keseluruhan, serta tampilkan juga biaya 10 epoch untuk model Neural Network, biaya 10 epoch untuk model Regresi Linier, dan biaya 10 epoch dengan ukuran batch 50 untuk model Regresi Linier.

Jawaban:

1. Unguided

   file: Model.java

```java
package com.example.enggar.unguided;

public class Model {
    private String namaModel;
    private int jumlahDataLatih;

    public Model(String namaModel, int jumlahDataLatih) {
        this.namaModel = namaModel;
        this.jumlahDataLatih = jumlahDataLatih;
    }
```

```
    public int getJumlahDataLatih() {
        return jumlahDataLatih;
    }

    public String getNamaModel() {
        return namaModel;
    }

    public void tampilkanInfo() {
        System.out.println("Model: " + namaModel + " | Data latih: "
+ jumlahDataLatih);
    }

    // Overloading tanpa parameter
    public int hitungBiaya() {
        return jumlahDataLatih * 2;
    }

    // Overloading dengan satu parameter (epoch)
    public int hitungBiaya(int epoch) {
        return jumlahDataLatih * epoch;
    }
}
```

file: ModelKlasifikasi

```
package com.example.enggar.unguided;

public class ModelKlasifikasi extends Model {
     private int jumlahKelas;

    public ModelKlasifikasi(String namaModel, int jumlahDataLatih,
int jumlahKelas) {
        super(namaModel, jumlahDataLatih);
        this.jumlahKelas = jumlahKelas;
    }

    @Override
    public void tampilkanInfo() {
        super.tampilkanInfo();
        System.out.println("Jumlah kelas: " + jumlahKelas);
    }
```

```
    @Override
    public int hitungBiaya() {
        return super.hitungBiaya() * jumlahKelas;
    }
}
```

file: ModelRegresi.java

```
package com.example.enggar.unguided;

public class ModelRegresi extends Model {
        private int jumlahFitur;

    public ModelRegresi(String namaModel, int jumlahDataLatih, int
jumlahFitur) {
        super(namaModel, jumlahDataLatih);
        this.jumlahFitur = jumlahFitur;
    }

    @Override
    public void tampilkanInfo() {
        super.tampilkanInfo();
        System.out.println("Jumlah fitur: " + jumlahFitur);
    }

    @Override
    public int hitungBiaya() {
        return super.hitungBiaya() + (jumlahFitur * 10);
    }

    // Method baru: dua parameter (epoch dan batch)
    public int hitungBiaya(int epoch, int batch) {
        return (getJumlahDataLatih() * epoch) / batch;
    }
}
```

file: ModelKalsifikasiDL.java

```
package com.example.enggar.unguided;

public class ModelKlasifikasiDL extends ModelKlasifikasi{
     private int jumlahLayer;
```

```
    public ModelKlasifikasiDL(String namaModel, int jumlahDataLatih,
int jumlahKelas, int jumlahLayer) {
        super(namaModel, jumlahDataLatih, jumlahKelas);
        this.jumlahLayer = jumlahLayer;
    }

    @Override
    public void tampilkanInfo() {
        super.tampilkanInfo();
        System.out.println("Jumlah layer: " + jumlahLayer);
    }

    @Override
    public int hitungBiaya() {
        return super.hitungBiaya() + (jumlahLayer * 200);
    }
}
```

file: Main.java

```
package com.example.enggar.unguided;

public class Main {
     public static void main(String[] args) {
        Model[] daftarModel = new Model[3];

        daftarModel[0] = new ModelRegresi("Regresi Linier", 500, 4);
        daftarModel[1] = new ModelKlasifikasi("Decision Tree", 800,
3);
        daftarModel[2] = new ModelKlasifikasiDL("Neural Network",
1000, 5, 3);

        int totalBiaya = 0;

        for (Model m : daftarModel) {
            m.tampilkanInfo();
            int biaya = m.hitungBiaya();
            System.out.println("Biaya komputasi: " + biaya);
            System.out.println("---");
            totalBiaya += biaya;
        }

        System.out.println("Total biaya: " + totalBiaya);
```

```
        ModelKlasifikasiDL nn = (ModelKlasifikasiDL) daftarModel[2];
        ModelRegresi regresi = (ModelRegresi) daftarModel[0];

        System.out.println("Biaya 10 epoch (Neural Network): " +
nn.hitungBiaya(10));
        System.out.println("Biaya 10 epoch (Regresi Linier): " +
regresi.hitungBiaya(10));
        System.out.println("Biaya 10 epoch batch 50 (Regresi Linier):
" + regresi.hitungBiaya(10, 50));
    }
}
```

Output :

```
nggar.unguided.Main'
Model: Regresi Linier | Data latih: 500
Jumlah fitur: 4
Biaya komputasi: 1040
---
Model: Decision Tree | Data latih: 800
Jumlah kelas: 3
Biaya komputasi: 4800
---
Model: Neural Network | Data latih: 1000
Jumlah kelas: 5
Jumlah layer: 3
Biaya komputasi: 10600
---
Total biaya: 16440
Biaya 10 epoch (Neural Network): 10000
Biaya 10 epoch (Regresi Linier): 5000
Biaya 10 epoch batch 50 (Regresi Linier): 100
PS C:\Users\ADVAN WORKPLUS\Documents\coolyeah\sems 5
```

Penjelasan :

Script ini mensimulasikan perhitungan biaya komputasi tiga model machine learning menggunakan konsep inheritance. Terdiri dari empat class: induk Model, serta turunannya ModelRegresi, ModelKlasifikasi, dan ModelKlasifikasiDeepLearning, yang dijalankan di Main. Kelas Model memiliki atribut nama dan data latih, serta overloading method hitung biaya (tanpa parameter dan dengan parameter epoch).

ModelRegresi menambahkan atribut fitur, meng-override method tampilkan info dan hitung biaya (ditambah fitur × 10), serta menambah versi ketiga hitung biaya dengan

dua parameter (epoch dan batch). ModelKlasifikasi menambahkan atribut jumlah kelas dan meng-override hitung biaya tanpa parameter (dikali jumlah kelas), tanpa meng-override versi epoch. ModelKlasifikasiDeepLearning menerapkan pewarisan berjenjang dengan mewarisi ModelKlasifikasi, menambahkan atribut layer, dan meng-override hitung biaya (ditambah layer × 200).

Di Main, ketiga objek (Regresi Linier, Decision Tree, Neural Network) disimpan dalam array bertipe kelas induk. Sebuah perulangan menampilkan informasi dan menghitung biaya tiap model secara polimorfik, lalu menampilkan totalnya (16440). Terakhir, program memanggil method hitung biaya dengan parameter epoch dan batch secara spesifik untuk membuktikan semua versi method berjalan sesuai rumus.

D. Kesimpulan

Kesimpulannya adalah modul 6 menunjukkan bahwa inheritance (pewarisan) memungkinkan sebuah kelas turunan (subclass) untuk mewarisi atribut dan method dari kelas induknya (superclass) guna menghindari duplikasi kode, di mana kelas turunan bersifat lebih spesifik. Dalam implementasinya, keyword super digunakan untuk mengakses anggota atau constructor dari kelas induk. Modul ini juga membedah dua konsep penting, yaitu overloading (membuat beberapa method dengan nama sama namun parameter berbeda dalam satu kelas) dan overriding (menulis ulang method milik kelas induk di kelas turunan dengan nama dan parameter yang sama). Pemahaman terhadap konsep inheritance, polymorphism, serta interaksi antara superclass dan subclass ini sangat penting agar program berorientasi objek yang dibangun menjadi lebih efisien, terstruktur, dan mudah untuk dikembangkan.